


# UNIVERSIDAD PRIVADA DE TACNA

## FACULTAD DE INGENIERÍA

## Escuela Profesional de Ingeniería de Sistemas

**"Plataforma web para la comercialización y revalorización de productos perecibles orientada al consumo sostenible en la provincia de Tacna"**

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

***Renzo Fernando Loyola Vilca*** ***(2021072615)***

***Brant Antony Chata Choque*** ***(2020067577)***

**Tacna – Perú**

***2026***

# 1. Introducción y metodología

El presente informe analiza la trazabilidad del Sprint 5 de ÓrbiKa, denominado “Cancelación controlada”, planificado del 31 de octubre al 2 de noviembre de 2026, con una capacidad estimada de 3 puntos de historia. Su objetivo es permitir que un comprador cancele un pedido elegible y que las unidades reservadas se devuelvan al inventario sin duplicar el stock, ni reactivar publicaciones que el vendedor ya retiró o que ya vencieron.

# 2. Alcance y compromiso del Sprint 5

| Campo | Definición según la planificación |
|---|---|
| Nombre | Sprint 5 — Cancelación controlada |
| Periodo | 31 de octubre – 2 de noviembre de 2026 |
| Capacidad | 3 puntos de historia |
| Objetivo | Permitir cancelar un pedido elegible y devolver las unidades al inventario sin duplicar stock. |
| Requisito comprometido | RF-17 |
| Dependencia | Depende del pedido creado en el Sprint 4 y de sus estados. |
| Entregable | Cancelación con estado trazable y reposición de inventario comprobada. |
| Criterio de cumplimiento | Un pedido en estado cancelable cambia a cancelado y repone la cantidad una sola vez; un pedido entregado no puede cancelarse. |
| Evidencia planificada | Estado del pedido y del stock antes y después de la cancelación, con prueba de un segundo intento sobre el mismo pedido. |
| Tareas y responsables | Renzo: aplicar las reglas de estado y la reposición de stock dentro de una operación segura. Brant: validar la cancelación antes de la entrega, el intento después de la entrega y la repetición de la cancelación. |

### Resumen de estado basado en la evidencia disponible

| Requisito | Evidencia en el código | Estado de trazabilidad |
|---|---|---|
| RF-17 — Cancelación controlada | cancelar_pedido_transaccion() bloquea la fila del pedido (FOR UPDATE), valida pertenencia y estado, exige motivo, cambia el pedido a cancelado y repone el stock en la misma transacción; es idempotente ante una segunda cancelación. | Implementado y verificado con pruebas reales contra la base de datos (PGlite); persisten dos vacíos de prueba a nivel de integración HTTP y de interfaz (ver sección 4). |

# 3. Requerimiento — RF-17: Cancelación controlada

| Elemento de trazabilidad | Análisis |
|---|---|
| SRS — definición | CU-17 “Cancelar pedido” (RF-17): el pedido debe pertenecer al comprador y estar en estado Creado o Preparando; al cancelarse queda en estado Cancelado, el stock se restituye una sola vez, y el pedido se excluye de ventas y cálculos de impacto. Recuperar el stock no vuelve a publicar un producto retirado ni habilita uno ya vencido. |
| Caso de uso | CU-17, disparado desde “Mis compras”. El flujo principal del SRS ya detalla 9 pasos y 7 alternativas (pedido no cancelable, cancelación repetida, motivo vacío, error durante la operación) — es uno de los casos de uso mejor detallados del documento. |
| Evidencia backend — ruta y controlador | backend/src/routes/pedido.routes.js expone PATCH /api/pedidos/:id/cancelar con requireAuth + requireRole(“comprador”), comentado en el código como CU-17. pedido.controller.js → cancelar() delega directamente al servicio. |
| Evidencia backend — servicio | pedido.service.js → cancelarPedido(pedidoId, actor, motivo, db): exige rol comprador, valida que el id sea un entero positivo, valida el motivo con validarCancelacion() (obligatorio, máximo 300 caracteres) y delega la operación completa a db.rpc(“cancelar_pedido_transaccion”, {...}); si la base de datos rechaza la operación, la envuelve en un error 409 genérico. |
| Evidencia backend — base de datos | supabase/migrations/003_srs_integrity_and_images.sql define cancelar_pedido_transaccion(): toma un bloqueo de fila sobre el pedido (SELECT ... FOR UPDATE), verifica que el comprador sea el dueño (“no te pertenece”), es idempotente si el pedido ya está cancelado (lo devuelve sin repetir la operación), rechaza estados no cancelables (“ya no puede cancelarse”), exige un motivo no vacío, actualiza el pedido a cancelado con motivo y fecha, repone el stock del producto (stock = stock + cantidad) y notifica al vendedor — todo dentro de una sola función transaccional. |
| Evidencia frontend | OrderActions.jsx usa accionesPermitidas(rol, estado) (order.helpers.js) para mostrar el botón “Cancelar pedido” solo cuando el rol es comprador y el estado es creado o preparando. Al confirmar, abre un modal accesible (PromptModal) que exige un motivo de texto y envía PATCH /pedidos/:id/cancelar; muestra una notificación de éxito o de error según la respuesta. |
| Pruebas reales ejecutadas | Backend (PGlite, base PostgreSQL real en memoria, database.test.js): cancelar dos veces repone el stock y genera un solo aviso — PASS; cancelar no reactiva una publicación retirada — PASS; cancelar un pedido antiguo de un producto vencido mantiene su indisponibilidad — PASS; no se puede cancelar un pedido listo para entrega — PASS; un comprador ajeno no puede cancelar — PASS. Backend (mock, pedido.service.test.js): el servicio delega correctamente a la función transaccional — PASS. Backend (validación): exige motivo de cancelación — PASS. Backend (contrato, security-migration.test.js): la cancelación y la restitución de stock ocurren dentro de la misma función — PASS. Frontend (order.helpers.test.js): el botón “cancelar” se habilita solo en los estados correctos y desaparece en el resto, incluido cancelado — PASS (10 casos). |
| Lo que NO está probado con evidencia real | No se encontró una prueba de integración HTTP (supertest) que ejercite la ruta PATCH /api/pedidos/:id/cancelar en sí; el archivo de pruebas HTTP (backend/test/http/app.test.js) cubre POST /pedidos y PATCH /pedidos/:id/estado, pero no /cancelar — la exigencia del rol comprador en esa ruta solo se verifica indirectamente. Tampoco existe una prueba de componente que confirme que OrderActions.jsx realmente oculta el botón o abre el modal en el navegador; solo se prueba la función pura accionesPermitidas(). |

| Elemento de trazabilidad | Análisis |
|---|---|
| Criterio de aceptación | Un pedido en estado cancelable cambia a cancelado y repone la cantidad una sola vez; un pedido entregado (o en adelante) no puede cancelarse; debe documentarse con el estado del pedido y del stock antes/después, y con la prueba de un segundo intento. |

***Estado de desarrollo: implementado; aceptación verificada con pruebas reales a nivel de servicio y de base de datos.***

*El código implementa correctamente el mecanismo central del requisito: bloqueo de fila, verificación de pertenencia y de estado, exigencia de motivo, transacción atómica de cancelación + reposición de stock, protección contra reactivaciones indebidas (publicación retirada o producto vencido) y comportamiento idempotente ante una segunda cancelación. Ocho de las ocho pruebas relacionadas con cancelación se ejecutaron y pasaron, varias de ellas contra una base de datos PostgreSQL real (PGlite). El SRS ya describe este caso de uso (CU-17) con un nivel de detalle alto, y el código observado coincide con esa narrativa en cada rama, incluida la nota sobre no reactivar productos retirados o vencidos al recuperar stock. Antes de declarar RF-17 completamente aceptado conviene añadir una prueba HTTP de la ruta /cancelar y una prueba de interfaz del componente OrderActions.*

## 4. Matriz de casos de prueba

Los siguientes casos derivan del criterio de cumplimiento de la planificación y de las ramas del flujo de CU-17 en el SRS. A diferencia de los Sprints 3 y 4, la mayoría de los casos de este sprint ya cuentan con una ejecución real documentada; los dos casos pendientes corresponden a niveles de prueba que el proyecto aún no cubre (integración HTTP e interfaz), no a comportamiento sin implementar.

| ID | Requisito | Prueba y resultado | Resultado |
|---|---|---|---|
| CP-S05-01 | RF-17 | Cancelar un pedido propio en estado creado o preparando lo cambia a cancelado y repone el stock exactamente una vez. | **PASS — evidencia real** |
| CP-S05-02 | RF-17 | Cancelar el mismo pedido una segunda vez no repone el stock de nuevo ni duplica el aviso al vendedor. | **PASS — evidencia real** |
| CP-S05-03 | RF-17 | Cancelar un pedido no reactiva una publicación que el vendedor ya retiró. | **PASS — evidencia real** |
| CP-S05-04 | RF-17 | Cancelar un pedido antiguo de un producto ya vencido no lo vuelve a poner disponible. | **PASS — evidencia real** |
| CP-S05-05 | RF-17 | No se puede cancelar un pedido en estado listo para entrega (ni en adelante). | **PASS — evidencia real** |
| CP-S05-06 | RF-17 | Un comprador no puede cancelar el pedido de otro comprador. | **PASS — evidencia real** |
| CP-S05-07 | RF-17 | La cancelación exige un motivo; confirmarla sin motivo es rechazada. | **PASS — evidencia real** |
| CP-S05-08 | RF-17 | El servicio de pedidos delega la cancelación a la función transaccional de base de datos. | **PASS — evidencia real** |
| CP-S05-09 | RF-17 | La cancelación y la restitución de stock ocurren dentro de una sola función transaccional (no en pasos separados). | **PASS — evidencia real** |

| ID | Requisito | Prueba y resultado | Resultado |
|---|---|---|---|
| CP-S05-10 | RF-17 | La interfaz solo ofrece “Cancelar pedido” en los estados creado y preparando, y la retira en el resto (incluido cancelado). | **PASS — evidencia real** |
| CP-S05-11 | RF-17 | La ruta PATCH /api/pedidos/:id/cancelar solo acepta al rol comprador (prueba de integración HTTP). | **Pendiente de ejecución** |
| CP-S05-12 | RF-17 | El componente OrderActions.jsx muestra el botón, abre el modal y envía la cancelación correctamente en el navegador (prueba de interfaz). | **Pendiente de ejecución** |

### Resultados reales de la ejecución de pruebas (02/11/2026)

| Suite | Archivos | Resultado |
|---|---|---|
| Backend — completo (con variables de entorno de prueba) | 10 archivos | 91 de 91 pruebas superadas |
| Backend — relacionadas con cancelación (filtro “cancel”) | 4 archivos | 8 de 8 pruebas superadas |
| Frontend — completo | 4 archivos | 49 de 49 pruebas superadas |
| Frontend — accionesPermitidas() (order.helpers.test.js) | 1 archivo | 10 de 10 pruebas superadas |

## 5. Verificación del Definition of Done

| Criterio | Estado | Evidencia / observación |
|---|---|---|
| Criterios de aceptación aceptados | Mayormente | Cancelación válida, idempotencia, bloqueo por estado no cancelable y por pertenencia ajena, y motivo obligatorio verificados con pruebas reales; falta la prueba HTTP del rol comprador en la ruta y la prueba de interfaz del componente. |
| Pruebas y resultado registrados | Sí | 140 pruebas reales ejecutadas en total (91 backend + 49 frontend); 8 de ellas específicas de cancelación. Resultados documentados en la sección 4. |
| Cambios integrados | Sí | El código de cancelación (ruta, controlador, servicio, validación y función SQL) está en la rama principal del repositorio. |
| Sin defectos críticos abiertos | No verificable | El repositorio no expone un registro de incidencias. |
| Evidencia guardada sin credenciales ni datos personales reales | Sí | Resultados de prueba y fragmentos de código citados en este informe; sin datos sensibles ni credenciales reales. |

## 6. Hallazgos y acciones recomendadas

| N.° | Hallazgo | Acción recomendada | Responsable |
|---|---|---|---|
| H-01 | No existe una prueba de integración HTTP para PATCH /api/pedidos/:id/cancelar; el archivo backend/test/http/app.test.js cubre POST /pedidos y PATCH /pedidos/:id/estado, pero no /cancelar. | Agregar un caso a app.test.js que confirme que un vendedor recibe 403 al intentar cancelar y que un comprador dueño del pedido recibe 200. | Brant |
| H-02 | No existe una prueba de componente para OrderActions.jsx; solo se prueba la función pura accionesPermitidas() que decide qué botones mostrar. | Agregar una prueba con Testing Library que verifique que el botón “Cancelar pedido” abre el modal y que confirmar con motivo llama a PATCH /pedidos/:id/cancelar. | Renzo |
| H-03 | Sin un archivo .env configurado, 3 de 10 suites del backend no cargan (mismo hallazgo que en los informes de Sprint 3 y 4). | Documentar variables de entorno de prueba (sin secretos reales) para CI, o versionar un .env.test de ejemplo. | Renzo |
| H-04 | El Sprint Backlog planificado no coincide con el estado real del repositorio: todo el código corresponde a un único commit que ya incluye funcionalidades de varios sprints, incluido RF-17. | Mantener commits incrementales por sprint en adelante para sustentar trazabilidad cronológica real. | Ambos |

## 7. Conclusiones

- RF-17 tiene una base sólida: la función cancelar_pedido_transaccion bloquea la fila del pedido, verifica pertenencia y estado cancelable, exige un motivo, cambia el pedido a cancelado y repone el stock dentro de una única transacción atómica.
- El comportamiento es idempotente ante una segunda cancelación (no repone el stock dos veces ni duplica el aviso al vendedor) y protege contra reactivaciones indebidas: cancelar un pedido no vuelve a publicar un producto retirado ni habilita uno ya vencido.
- De los doce casos de prueba identificados para este sprint, diez cuentan con evidencia real de ejecución (PASS) y dos permanecen pendientes por tratarse de niveles de prueba que el proyecto aún no cubre: integración HTTP de la ruta y prueba de interfaz del componente, no comportamiento sin implementar.
- El caso de uso CU-17 del SRS ya describe este requisito con un nivel de detalle alto (9 pasos en el flujo principal y 7 alternativas), y el código observado coincide con esa narrativa en cada rama revisada, incluida la nota sobre no reactivar productos retirados o vencidos al recuperar stock.
- En conjunto, 140 pruebas automatizadas reales se ejecutaron entre backend y frontend sin fallos, de las cuales 8 verifican específicamente el comportamiento de cancelación; la limitación de entorno (.env) señalada en los informes de Sprint 3 y 4 se mantiene como pendiente.
- El objetivo central del Sprint 5 —cancelación con reposición de stock trazable y sin duplicados— se cumple en el código; para cerrar completamente el Definition of Done faltan dos pruebas por agregar (HTTP e interfaz), no cambios de funcionalidad.

## 8. Bibliografía y fuentes técnicas


- Grupo G01. (2026). Planificación de Sprints de ÓrbiKa, versión 1.0, 12 de octubre de 2026: Sprint Backlog, sprints 5-6.
- Grupo G01. (2026). Informe de Trazabilidad — Sprint 4 ÓrbiKa (documento de referencia para el formato de este informe).

- Grupo G01. (2026). Especificación de requisitos de software (SRS), RF-17 y CU-17.
- Repositorio ORBIKA. (2026). supabase/migrations/001_init_schema.sql, 002_security_and_order_transactions.sql y 003_srs_integrity_and_images.sql.
- Repositorio ORBIKA. (2026). backend/src/services/pedido.service.js, backend/src/validation/pedido.validation.js, backend/src/controllers/pedido.controller.js, backend/src/routes/pedido.routes.js.
- Repositorio ORBIKA. (2026). backend/test/sql/database.test.js, backend/test/sql/security-migration.test.js, backend/test/services/pedido.service.test.js, backend/test/validation/pedido.validation.test.js, backend/test/http/app.test.js.
- Repositorio ORBIKA. (2026). frontend/src/components/OrderActions.jsx, frontend/src/pages/order.helpers.js, frontend/src/pages/order.helpers.test.js.


*Repositorio: https://github.com/renzoloyola/G01_CSW_ORBIKA*